Build Your First Library

The path to PyPI

01

Function

pandas code

02

Package

one folder

03

Build

make a wheel

04

Publish

pip install

Today's goals

01 Package

Create a real, importable Python package.

02 Publish

Upload it to TestPyPI, then PyPI.

03 Install

pip install your own package.

Build tiny. Publish safely.

What we're building

```
import pandas as pd
import simple_eda as eda

df = pd.read_csv("data.csv")
eda.summarize(df)
eda.missing(df)
```

What we build now — and skip

Build now

One dependency: pandas

Plain Python functions

A clear README

Skip for now

CI/CD

Testing

Many dependencies

01

What is a Python library?

Library

Reusable code you install once and import anywhere.

Script vs library

Script

python analyze.py

Less convenient

Limited scope

Library

import simple_eda

More convenient

Broad scope

A tiny library is still real

No web app required.

No dashboard required.

Just useful, installable, importable.

The five bare minimums

A unique package name

A project folder

An __init__.py file

Your functions

Metadata — pyproject.toml

02

Design the smallest API

API

The functions and classes users call.

MVP

The smallest version that is genuinely useful.

the simplest API

```
import simple_eda as eda
eda.summarize(df)
```

Pick boring function names

Prefer (boring)	Avoid (clever)
summarize	hyper_profile_engine
missing	magic_diagnose
numeric_columns	mega_summary

MVP Input: Pandas dataframe only

Support pandas DataFrames first.

Not Polars, Spark, Dask, or Arrow yet.

Fewer choices means fewer bugs.

MVP Output: Plain objects only

Dictionaries & lists

Simple built-in containers.

Ints & strings

Plain scalar values.

pandas Series

No custom classes to learn.

The 0.1.0 mindset

First working version, not perfect.

Installable and understandable.

Momentum, not completeness.

03

Create the package folder

Final folder shape

```
simple-eda-project/
├── src/
│   └── simple_eda/
│       ├── __init__.py
│       └── core.py
├── README.md
├── pyproject.toml
└── LICENSE
```

Install name vs import name

Install name	Import name
simple-eda-you	simple_eda
hyphens	underscores
used by pip & PyPI	used in Python code

Why the names differ

import needs a valid Python name.

Python names cannot contain hyphens.

So the import name uses underscores.

PyPI treats - and _ as the same name.

__init__.py

Marks a folder as an importable package.

Why the src/ layout?

Blocks accidental root imports.

Matches modern Python projects.

Makes packaging behavior clear.

Common folder mistakes

pyproject.toml in the wrong folder.

Forgetting __init__.py.

Folder named simple-eda, not simple_eda.

Installing from the wrong directory.

What each mistake causes

Mistake	What happens
No __init__.py	import cannot find it
Hyphen in folder	invalid module name
pyproject misplaced	build finds nothing
Wrong directory	ModuleNotFoundError

04

Write the EDA functions

```
def summarize(df):
    return {
        "rows": len(df),
        "columns": len(df.columns),
        "names": list(df.columns),
    }
```

The core functions

summarize — shape, columns, types

missing — nulls per column

numeric_columns / categorical_columns

missing()

```
def missing(df):
    counts = df.isna().sum()
    return counts.sort_values(
        ascending=False)

# missing(df) ->
# age     2
# name    1
```

numeric & categorical columns

```
def numeric_columns(df):
    sel = df.select_dtypes(
        include="number")
    return list(sel.columns)

def categorical_columns(df):
    sel = df.select_dtypes(
        include=["object", "category"])
    return list(sel.columns)
```

Try it on a tiny DataFrame

```
import simple_eda as eda

df = pd.DataFrame({
    "name": ["Ana", "Bo", None],
    "age": [25, None, 31],
})
print(eda.summarize(df))
print(eda.missing(df))
```

Function do's and don'ts

Do	Don't
Return data	Change df in place
Use plain dicts / Series	Print instead of return
Keep signatures small	Add many optional args
One job per function	One function for everything

One job vs everything

One job each	One big function
Small and focused	Hard to follow
Easy to test	Hard to test
Easy to reuse	Breaks more often

Signature

A function name plus the parameters it takes.

05

Add pyproject.toml

What pyproject.toml does

Names your package and version.

Lists dependencies.

Tells build tools how to build.

Build tools

Tools that turn source into installable packages.

Minimal pyproject.toml

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "simple-eda-yourname"
version = "0.1.0"
description = "Tiny EDA helper"
dependencies = ["pandas>=1.5"]
```

TOML

The simple key = value format pyproject uses.

Version numbers

Version	Means
0.1.0	First working version
0.1.1	A small fix
0.2.0	A new feature
1.0.0	Stable and public

MAJOR.MINOR.PATCH

Each of the three numbers is a non-negative integer.

README: install & use

simple-eda-yourname
Tiny EDA helper for pandas.

Install
pip install simple-eda-yourname

Use
import simple_eda as eda
print(eda.summarize(df))

License basics

Says what others may do with your code.

MIT is common for small projects.

Use an official template.

Never invent your own license text.

06

Install locally with pip

Install in editable mode

```
# run from the folder with pyproject.toml
python -m pip install -e .
```

What -e (editable) means

Links pip to your source folder.

Code edits apply instantly.

No reinstall while developing.

Smoke-test the import

```
>>> import simple_eda as eda
>>> eda.__all__
['summarize', 'missing', ...]
```

When the import fails

Symptom	Fix
ModuleNotFoundError	Run pip install -e .
Wrong folder	cd to the project root
Wrong environment	Use python -m pip

07

Build a real package file

Build the artifacts

```
python -m pip install build
python -m build
```

The dist/ folder

dist/
■■■ simple_eda_yourname-0.1.0.tar.gz
■■■ simple_eda_yourname-0.1.0-...whl

Wheel vs source distribution

Source (.tar.gz)

A source archive

Used as a fallback

Wheel (.whl)

Prebuilt, fast install

What most users get

Keep build output out of git

dist/, build/, *.egg-info are generated.

Add them to .gitignore.

Bump the version before rebuilding.

08

Publish safely

Why TestPyPI first?

A practice package index.

Rehearse upload and install commands.

Mistakes are cheap here.

The upload flow

01 Check

python -m twine check dist/*

02 TestPyPI

twine upload --repository testpypi dist/*

03 PyPI

twine upload dist/* — only after TestPyPI works.

Install from TestPyPI

```
# install from TestPyPI
pip install \
  -i https://test.pypi.org/simple/ \
  --extra-index-url \
    https://pypi.org/simple/ \
  simple-eda-yourname
```

Publishing safety

Use API tokens, not passwords.

Never commit tokens or secrets.

Fewer deps, less attack surface.

Beware typosquatting.

09

Use, improve & share

What to add — and avoid

Avoid early

Full HTML reports

Plugin systems

Every dataframe backend

AI-generated analysis

Good next steps

quick_report()

top_categories()

column_summary()

duplicate_rows()

Add a tiny test

```
def test_numeric_columns():
    df = pd.DataFrame(
        {"name": ["A"], "age": [10]})
    cols = eda.numeric_columns(df)
    assert cols == ["age"]
```

Final checklist

Package files all exist.

pip install -e . works.

python -m build works.

twine check dist/* passes.

README has install + usage.

0.1.0

Ship your first version

Four functions, importable, published.